

WEEK 2 - LECTURE 2

Computer Vision for Warehouse Robots

Giving Robots the Gift of Sight

Course: Introduction to Robotics for AI and Data Science

Instructor: Dr. Akram Fatayer

[linkedin.com/in/akram-fatayer](https://www.linkedin.com/in/akram-fatayer)

Institution: Al-Zaytoonah University of Jordan



The Warehouse Robot Challenge

Your warehouse robot has a mission: locate and retrieve a specific red box among hundreds of items.

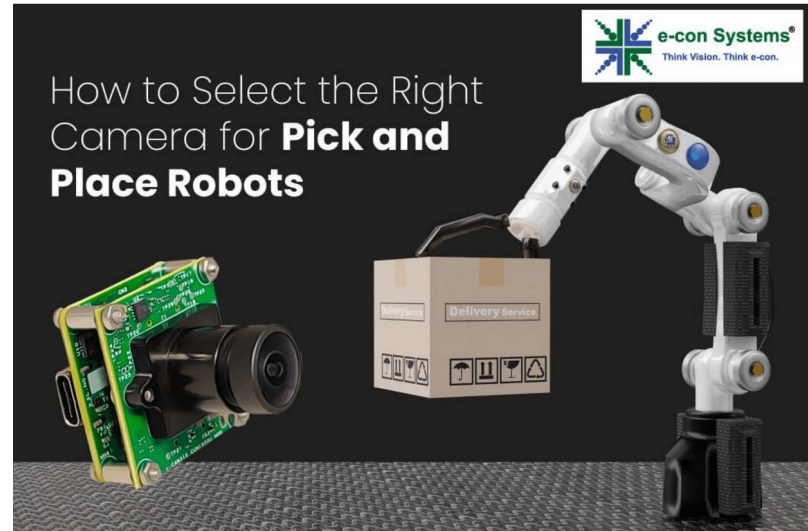
Human Approach

Look around the warehouse, visually scan for red objects, recognize the target box, navigate to it.

Current Robot

Can move around the warehouse, but cannot see or identify objects. It's essentially blind.

The Problem: How do we give the robot the ability to perceive and understand its visual environment?



Why Computer Vision is Critical for Robotics

\$214B

Global Robotics Market by 2030

80%

Of Robots Need Vision Systems

\$50B+

Warehouse Automation Market

Real-World Applications



Warehouse Automation

Amazon, FedEx: Item picking, sorting, inventory management



Autonomous Vehicles

Tesla, Waymo: Object detection, lane keeping, navigation



Manufacturing

Quality inspection, assembly guidance, defect detection



Agriculture

Crop monitoring, fruit picking, precision farming

Breaking Down the Vision Problem

Complex problems become manageable when decomposed into steps

1

Image Capture

Get raw pixels from camera sensor

2

Image Processing

Filter, enhance, transform color spaces

3

Object Detection

Find regions containing objects

4

Recognition

Classify what each object is

5

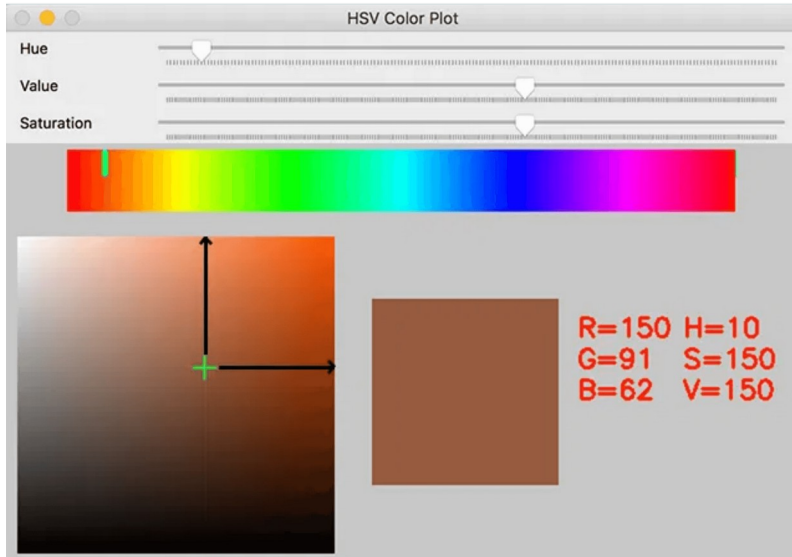
3D Localization

Estimate position in 3D space



Each step uses techniques you already know from AI/DS: data preprocessing, feature extraction, classification, regression

Cameras as Data Sensors



1. Lens System

Focuses incoming light rays from the 3D world onto a 2D sensor plane. Controls field of view and depth of field.

2. Sensor Array

Grid of photodetectors (pixels) that convert photons into electrical signals. Each pixel measures light intensity.

3. Color Filters

Bayer filter pattern separates light into Red, Green, Blue channels. Three measurements per pixel location.

Data Output

Image: $H \times W \times 3$ array

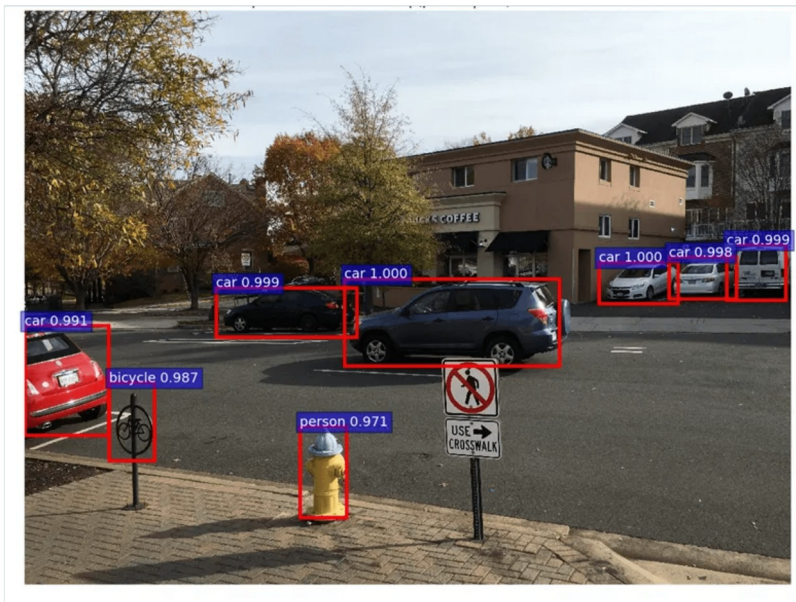
Camera Parameters and Trade-offs

Choosing the right camera configuration for robotic applications

Parameter	Description	Benefit	Trade-off
Resolution	Number of pixels (e.g., 640×480, 1920×1080)	Higher detail, better object recognition	More computation, slower processing
Frame Rate	Images per second (e.g., 30 FPS, 60 FPS)	Smoother tracking, faster response	Higher bandwidth, more processing load
Field of View	Angular extent of observable world (narrow vs. wide)	Wide: see more area Narrow: more detail	Wide: distortion Narrow: limited context
Focal Length	Distance from lens to sensor (short vs. long)	Long: zoom capability Short: wider view	Long: narrow FOV Short: less detail

For warehouse robots: Balance is key — 640×480 at 30 FPS with moderate FOV provides good performance without overwhelming the processor

From 3D World to 2D Image



Pinhole Camera Model

Light rays from 3D points pass through a single point (optical center) and project onto the image plane. This creates perspective projection.

Perspective Projection

Objects farther away appear smaller. Parallel lines converge to vanishing points. This is how humans see the world too.

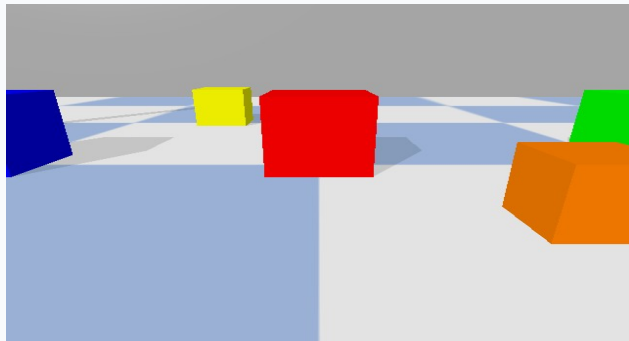
$$x_{\text{image}} = f \times (X_{\text{world}} / Z_{\text{world}})$$

The Fundamental Challenge

Depth information is lost! A 2D point in the image could correspond to any 3D point along a ray. We need additional

Images as Data: The AI/DS Perspective

Images are just structured numerical arrays — familiar territory for data scientists



Array Representation

```
shape: (480, 640, 3)
dtype: uint8 (0-255)
size: 921,600 values
```

```
# Load image as NumPy array
import cv2
import numpy as np

img = cv2.imread('robot_view.jpg')
print(img.shape) # (480, 640, 3)

# Access pixel at row 100, col 200
pixel = img[100, 200] # [R, G, B]

# Slice region of interest
roi = img[100:200, 150:250]

# Apply operations (just like any array)
mean_color = np.mean(img, axis=(0,1))
```

Key Insights

Images are 3D arrays: Height × Width × Channels

Use familiar tools: NumPy, PyTorch, Pandas

Color Spaces: RGB vs HSV

RGB Color Space

Red, Green, Blue — How cameras capture light

Three channels: R, G, B (0-255 each)

Mixes color and brightness together

Same red appears different in shadow vs. sunlight

Hard to define "red" with simple thresholds

Problem: Sensitive to lighting variations

HSV Color Space

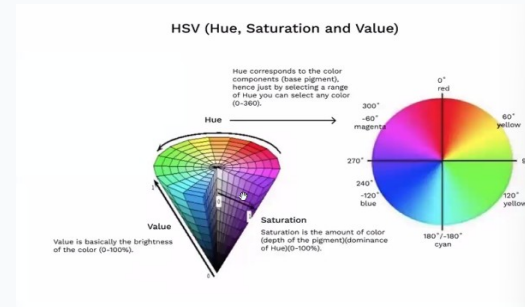
Hue, Saturation, Value — How humans perceive color

Hue: The actual color (0-180°)

Saturation: Color intensity (0-255)

Value: Brightness (0-255)

Separates color from lighting conditions



Why HSV for Robotics?

Warehouse lighting changes throughout the day. HSV allows us to detect "red objects" regardless of whether they're in bright light or shadow — just threshold the Hue channel!

Color-Based Detection Pipeline

1

RGB → HSV

Convert image to HSV color space

2

Threshold

Create binary mask for target color range

3

Morphology

Clean noise with erosion and dilation

4

Find Contours

Detect connected regions in mask

5

Bounding Box

Draw rectangles around detected objects

```
# Complete color detection pipeline
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, lower_red, upper_red)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
# Draw bounding boxes
for contour in contours:
    x, y, w, h = cv2.boundingRect(contour)
    cv2.rectangle(image, (x,y), (x+w,y+h), (0,255,0), 2)
```

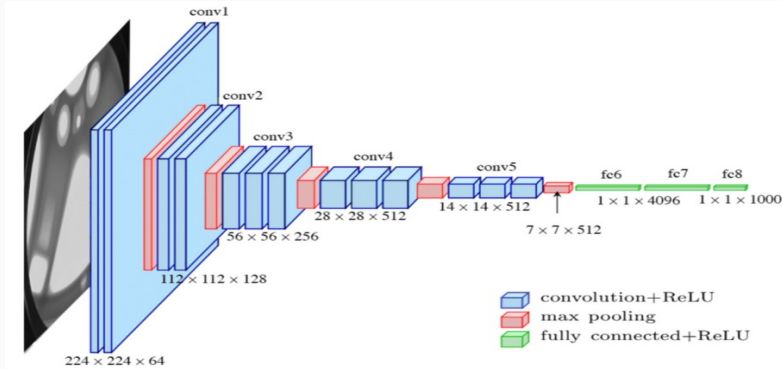
✓ Advantages

- Fast and simple to implement
- Works well for distinct colors
- No training data needed

✗ Limitations

- Sensitive to lighting changes
- Cannot recognize object types
- Fails with similar colors

CNNs Learn Visual Features



Hierarchical Feature Learning

Layer 1
Edges

Layer 2-3
Textures

Layer 4-5
Parts

Layer 6+
Objects

Why CNNs Work

CNNs automatically learn relevant features from data instead of requiring hand-crafted feature engineering. Early layers detect simple patterns (edges), deeper layers combine them into complex representations (objects).

Convolutional Layers

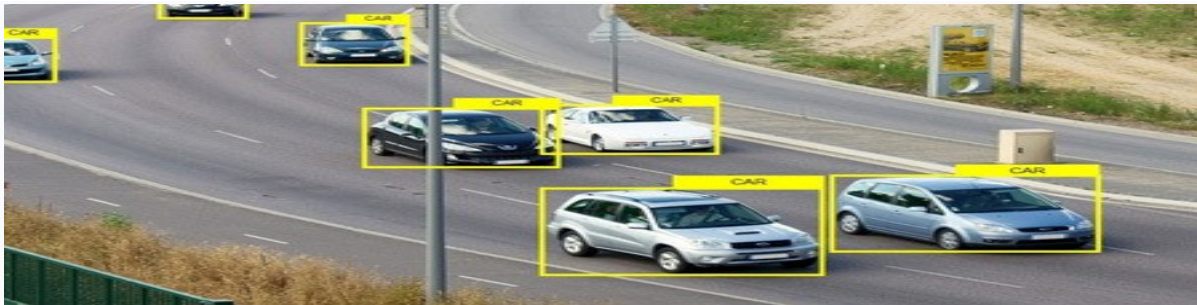
Apply learned filters across the image. Each filter detects a specific pattern. Spatial structure is preserved, unlike fully-connected layers.

Pooling Layers

Reduce spatial dimensions while keeping important features. Provides translation invariance — object detected regardless of exact position.

Connection to Your ML Knowledge

Object Detection for Robotics



Two-Stage Detectors

Examples: R-CNN, Faster R-CNN

Step 1: Propose regions

Step 2: Classify each region

Higher accuracy

Slower inference (2-5 FPS)

Accuracy

High

Speed

Slow

One-Stage Detectors

Examples: YOLO, SSD, MobileNet

Single pass through network

Predict boxes and classes together

Good accuracy

Fast inference (30+ FPS)

Accuracy

Good

Speed

Fast

For Warehouse Robots: Use One-Stage Detectors

Real-time performance is critical. We'll use **MobileNet-SSD** (lightweight, fast) and **YOLO** (more accurate) in the lab. Both run at 30+ FPS on modest hardware.

Transfer Learning: From COCO to Warehouse



Pre-training

Model trained on large dataset (ImageNet: 1M images, COCO: 330K images) learns general visual features



Transfer

Use pre-trained weights as starting point. Early layers (edges, textures) transfer well across domains



Fine-tuning (Optional)

Train on warehouse-specific data to adapt to your domain. Only needed if pre-trained model insufficient



Why Transfer Learning?

- No need to collect millions of training images
- Faster training (hours instead of weeks)
- Better performance with limited data
- Leverages knowledge from large datasets

</> In Practice

- Download pre-trained MobileNet-SSD or YOLO
- Use directly for common objects (boxes, people)
- Fine-tune if detecting custom objects
- Same concept from your ML course!

Deploying CNNs on Robots

Robots have limited compute power, memory, and battery — we need efficient models

Model	Size	Speed (FPS)	Accuracy (mAP)	Use Case
ResNet-101	170 MB	5 FPS	76%	Cloud/Server
YOLOv5	28 MB	45 FPS	65%	Desktop/Laptop
MobileNet-SSD	6 MB	60 FPS	60%	Embedded/Robot
TinyYOLO	4 MB	80 FPS	55%	Microcontroller

Efficient Architectures

MobileNet, EfficientNet use depthwise separable convolutions to reduce parameters while maintaining accuracy

Model Quantization

Convert 32-bit floats to 8-bit integers. 4× smaller, 2-4× faster, minimal accuracy loss

Edge AI Accelerators

Google Coral TPU, NVIDIA Jetson, Intel Neural Compute Stick provide hardware acceleration

Object Tracking for Navigation

Tracking maintains object identity across frames — more efficient than detecting from scratch each time

Detection Only

Run CNN on every frame independently. No memory of previous frames.

Pros: Simple, no state management

Cons: Computationally expensive, no temporal consistency, ID switching

Detection + Tracking

Detect occasionally, track between detections. Predict object location based on motion.

Pros: Faster, stable IDs, motion prediction

Cons: More complex, can lose track

Centroid Tracking

Match objects by distance between centroids in consecutive frames. Simple but effective.

Use: Slow-moving objects

Kalman Filter

Predict next position using motion model. Optimal for linear motion with Gaussian noise.

Use: Predictable trajectories

DeepSORT

Combines appearance features (CNN) with motion (Kalman). Handles occlusions well.

Use: Complex scenarios

Warehouse Robot Application

Track target item as robot approaches. Detect every 10 frames, track in between. Predict item location if temporarily occluded. Maintain stable ID for grasping system.

Computer Vision as Machine Learning

Generic ML

Data

Tabular, text, time series

Preprocessing

Normalization, encoding

Model

Neural net, tree, SVM

Evaluation

Accuracy, F1, AUC

Computer Vision

Data

Images ($H \times W \times 3$ arrays)

Preprocessing

Resize, augment, normalize

Model

CNN (ResNet, YOLO)

Evaluation

mAP, IoU, precision

Same Concepts, Different Data Type

Everything you learned in ML applies to CV: train/val/test splits, overfitting, regularization, transfer learning, hyperparameter tuning. Images are just high-dimensional structured data!

Data

NumPy arrays, PyTorch tensors

Training

Same optimizers (Adam, SGD)

Loss

Cross-entropy, MSE, IoU

Frameworks

PyTorch, TensorFlow, Keras

Sim-to-Real Gap in Vision

Simulation (PyBullet)

Perfect lighting, no shadows
Clean textures, no noise
Ideal camera (no distortion, blur)
Consistent colors
No occlusions or clutter

Real World

Variable lighting, shadows, reflections
Sensor noise, motion blur
Lens distortion, chromatic aberration
Color variations (manufacturing, wear)
Background clutter, occlusions

Bridging the Gap

Domain Randomization

Vary lighting, textures, colors in simulation. Train on diverse conditions so model generalizes better to real world.

Realistic Rendering

Use physics-based rendering with shadows, reflections, noise. Makes simulation closer to reality.

Real-World Fine-tuning

Train in sim, collect small real dataset, fine-tune. Combines sim efficiency with real-world accuracy.

In This Course

Weeks 1-3: Develop algorithms in PyBullet simulation. Week 4+: Test on real hardware (Raspberry Pi camera, Arduino). Experience the sim-to-real challenge firsthand and learn to adapt!

Week 2 Lab: Computer Vision for Warehouse Robot

1

Camera Setup

Configure virtual camera in PyBullet. Capture and display RGB images from robot's perspective.

25 points

2

Color Detection

Implement HSV-based color detection. Detect and localize colored objects in warehouse scene.

30 points

3

CNN Detection

Use MobileNet-SSD and YOLO for object detection. Compare performance and accuracy.

30 points



Deliverables

- Python code files (all parts)
- Screenshots showing detection results
- Lab report (3-5 pages) answering questions
- Comparison analysis (color vs CNN)



Timeline & Support

- Due:** Next class (1 week)
- Lab Hours:** TBD for hands-on help
- Starter Code:** Available on GitHub
- Office Hours:** For questions and debugging

Warehouse Robot Project Progress



Module 1: Basics

Robot simulation, motion control, sensor reading

✓ COMPLETED



Module 2: Vision

Camera setup, object detection, visual perception

▶ CURRENT

Module 3: LIDAR

Distance sensing, obstacle detection, mapping

UPCOMING



Module 4: Navigation

Path planning, obstacle avoidance, autonomous movement

UPCOMING



Capability Added This Week

Your warehouse robot can now **see and recognize objects** using its camera! It can detect items by color or use deep learning to identify specific object types. This is the foundation for autonomous item retrieval.

Next Week: LIDAR and Sensor Fusion

We'll add distance sensing to complement vision. Learn how to combine camera and LIDAR data for robust perception in cluttered warehouse environments.

Key Takeaways

1

Computer vision is critical for autonomous robots

80% of robots need vision systems. Cameras provide rich information about the environment for perception and decision-making.

2

Break complex problems into manageable steps

Vision pipeline: capture → process → detect → recognize → localize. Each step builds on the previous one.

3

HSV is better than RGB for robotic applications

HSV separates color from brightness, making detection robust to lighting variations in real-world environments.

4

CNNs learn hierarchical visual features automatically

Early layers detect edges, deeper layers combine them into objects. Transfer learning lets us use pre-trained models.

5

Real-time performance requires efficient models

MobileNet and YOLO provide good accuracy at 30+ FPS. Trade-offs between model size, speed, and accuracy.

6

Computer vision is just another ML problem

Same concepts you know: training, validation, overfitting, transfer learning. Images are structured data arrays!

Questions?



Email: a.fatayer@zuj.edu.jo



Office Hours: email me to book a meeting



[linkedin.com/in/akram-fatayer](https://www.linkedin.com/in/akram-fatayer)

Thank you!

Dr. Akram Fatayer • AI and Data Science Department
Al-Zaytoonah University of Jordan